Meet-in-the-Middle Attacks on Full ChiLow

Eran Lambooij[1] Patrick Neumann[1] Michiel Verbauwhede[2] Shichang Wang[3] Tianyu Zhang[3]

[1]Inria, Paris, France

[2]COSIC, KU Leuven, Leuven, Belgium

[3]Nanyang Technological University, Singapore

ChiLow

- Tweakable block cipher

ChiLow

- Tweakable block cipher
- Published at Eurocrypt 2025
- Designed by Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche and Damian Vizár

ChiLow

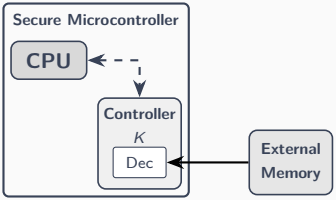
- Tweakable block cipher
- Published at Eurocrypt 2025
- Designed by Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche and Damian Vizár
- Versions: ChiLow-n, block size $n \in \{32, 40\}$

ChiLow

- Tweakable block cipher
- Published at Eurocrypt 2025
- Designed by Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche and Damian Vizár
- Versions: ChiLow-n, block size $n \in \{32, 40\}$
- 128-bit key & 64 bit tweak

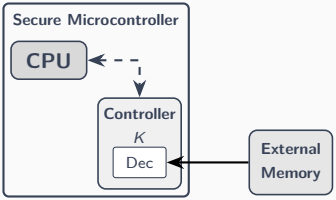
ChiLow

- Tweakable block cipher
- Published at Eurocrypt 2025
- Designed by Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche and Damian Vizár
- Versions: ChiLow-n, block size $n \in \{32, 40\}$
- 128-bit key & 64 bit tweak
- Application: embedded code encryption

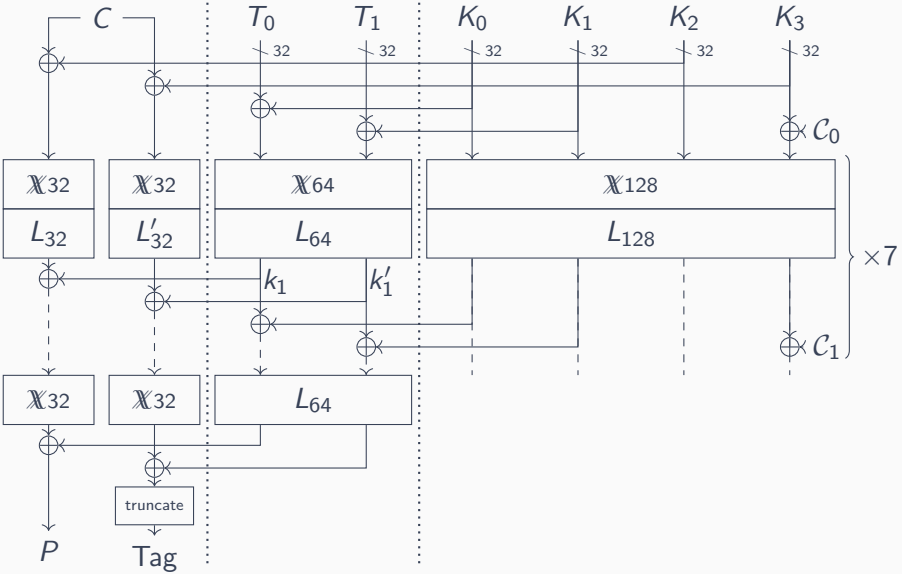


ChiLow

- Tweakable block cipher
- Published at Eurocrypt 2025
- Designed by Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche and Damian Vizár
- Versions: ChiLow-n, block size $n \in \{32, 40\}$
- 128-bit key & 64 bit tweak
- Application: embedded code encryption
- Decryption needs to be low latency



ChiLow-32


C
T0
T1
K0
K1
K2
K3
32
C0
X32
X32
X64
X128
L32
L'32
L64
L128
×7
k1
k'1
C1
X32
X32
L64
truncate
P
Tag

Meet-in-the-Middle Attacks on ChiLow

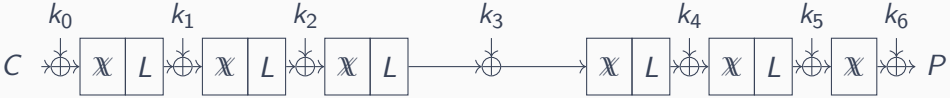
- Already considered by the designers

Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds

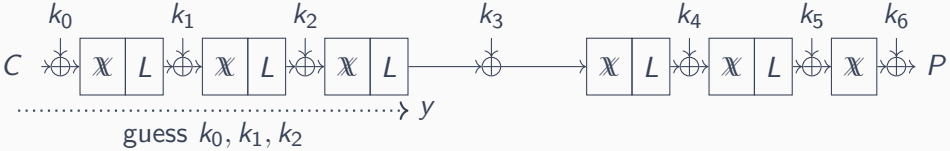
Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds



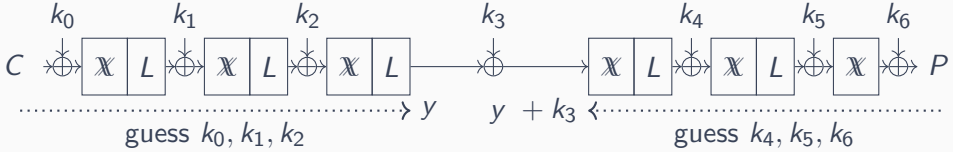
Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds



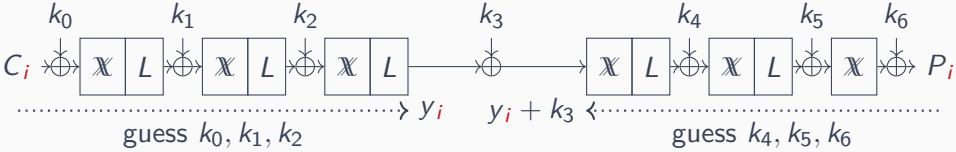
Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds



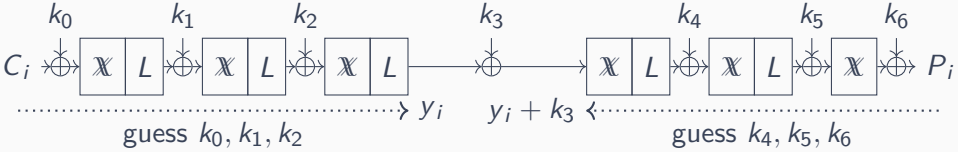
Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds



Meet-in-the-Middle Attacks on ChiLow

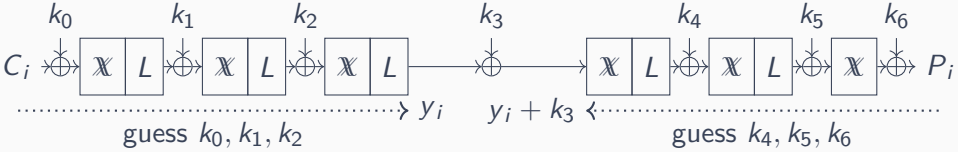
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$

Meet-in-the-Middle Attacks on ChiLow

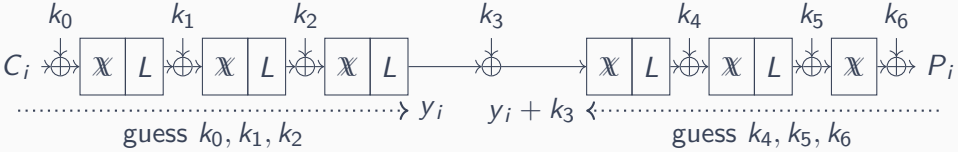
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $d2^{3n} + d2^{3n} + 2^{2 \cdot 3n - (d-1)n} \cdot 2^{64}$ $\quad (n \in \{32, 40\})$

Meet-in-the-Middle Attacks on ChiLow

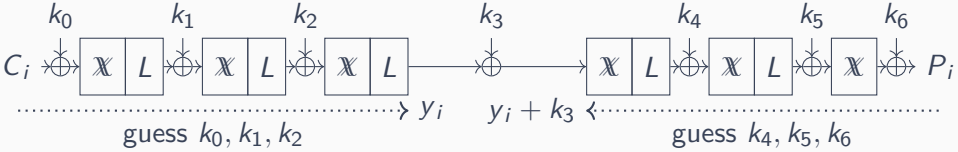
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $d2^{3n} + d2^{3n} + 2^{2\cdot 3n-(d-1)n} \cdot 2^{64}$ $\quad (n \in \{32, 40\})$

Meet-in-the-Middle Attacks on ChiLow

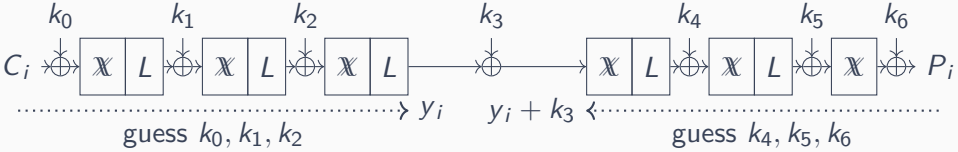
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $d2^{3n} + d2^{3n} + 2^{2\cdot 3n-(d-1)n} \cdot 2^{64}$ $\quad (n \in \{32, 40\})$

Meet-in-the-Middle Attacks on ChiLow

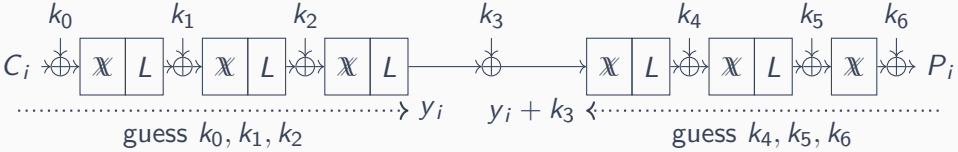
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $d2^{3n} + d2^{3n} + 2^{2\cdot 3n-(d-1)n} \cdot 2^{64}$ $\quad (n \in \{32, 40\})$

Meet-in-the-Middle Attacks on ChiLow

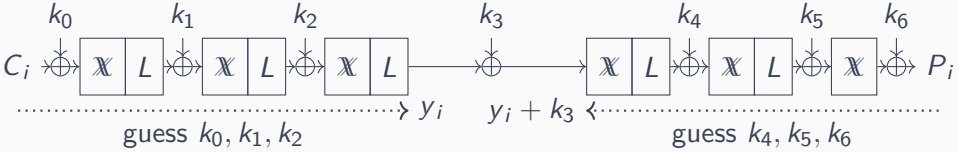
- Already considered by the designers
- Attacks found for up to 4 rounds



- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $d2^{3n} + d2^{3n} + 2^{2 \cdot 3n - (d-1)n} \cdot 2^{64}$ $\quad (n \in \{32, 40\})$

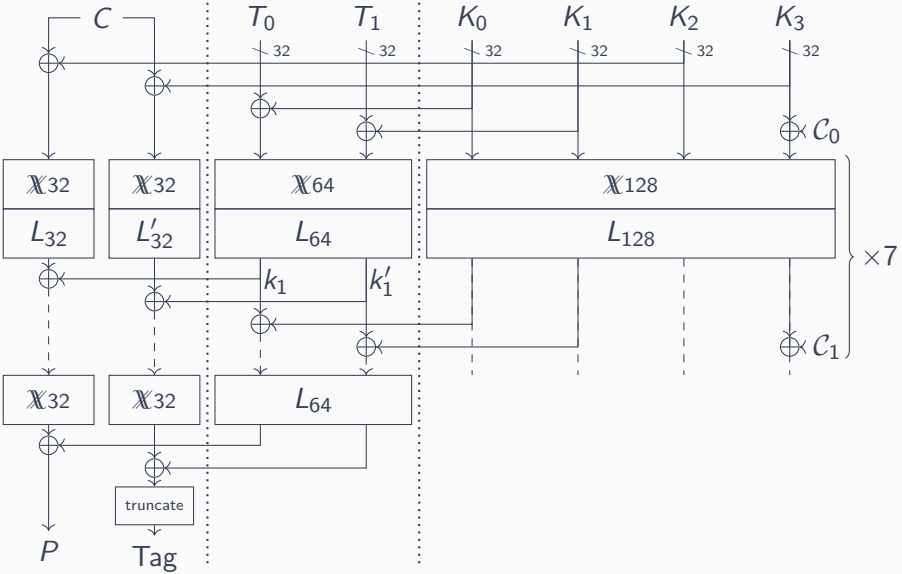
Meet-in-the-Middle Attacks on ChiLow

- Already considered by the designers
- Attacks found for up to 4 rounds

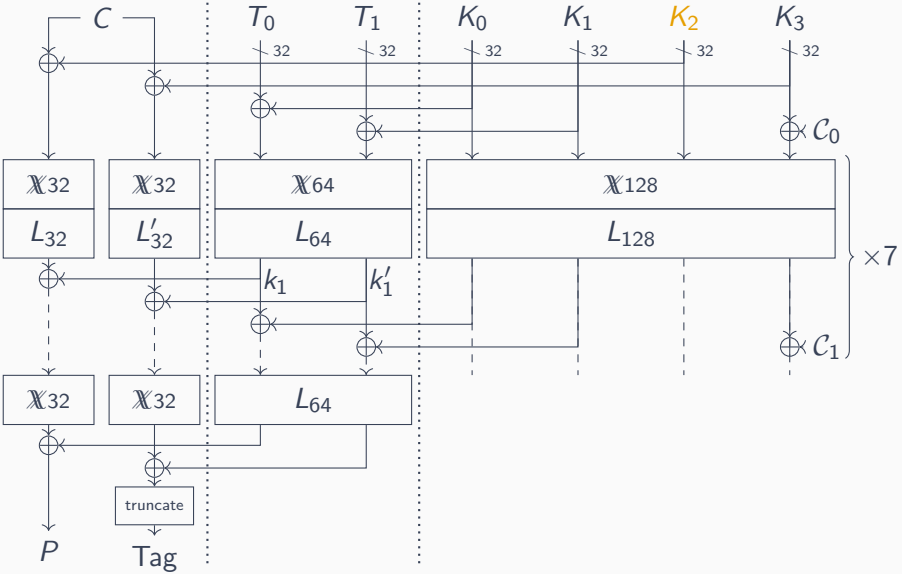


- Match on difference $y_i + y_0 = (y_i + k_3) + (y_0 + k_3)$
- Time complexity using d pairs: $\underbrace{d2^{3n} + d2^{3n} + 2^{2\cdot 3n-(d-1)n} \cdot 2^{64}}_{<2^{128} \text{ for } d=6}$ $\quad (n \in \{32, 40\})$

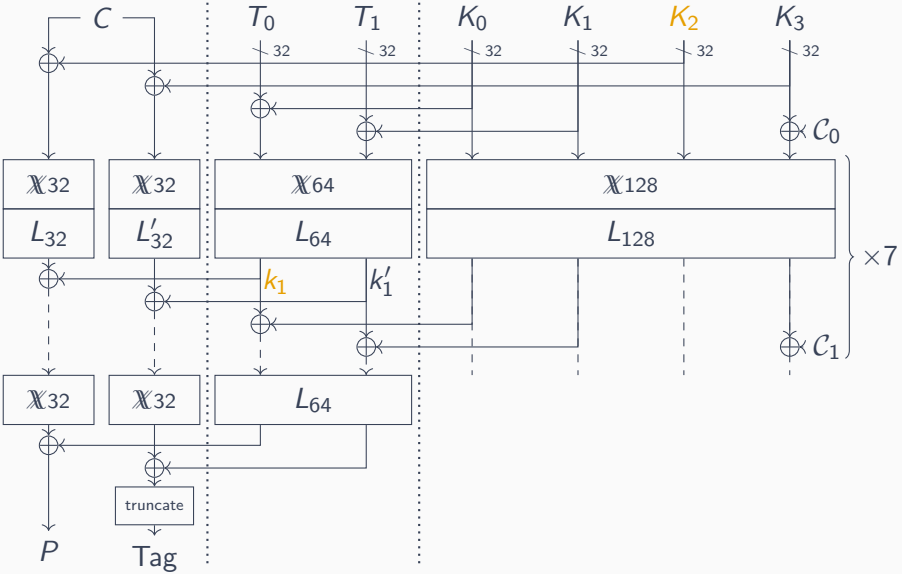
Master Key Recovery



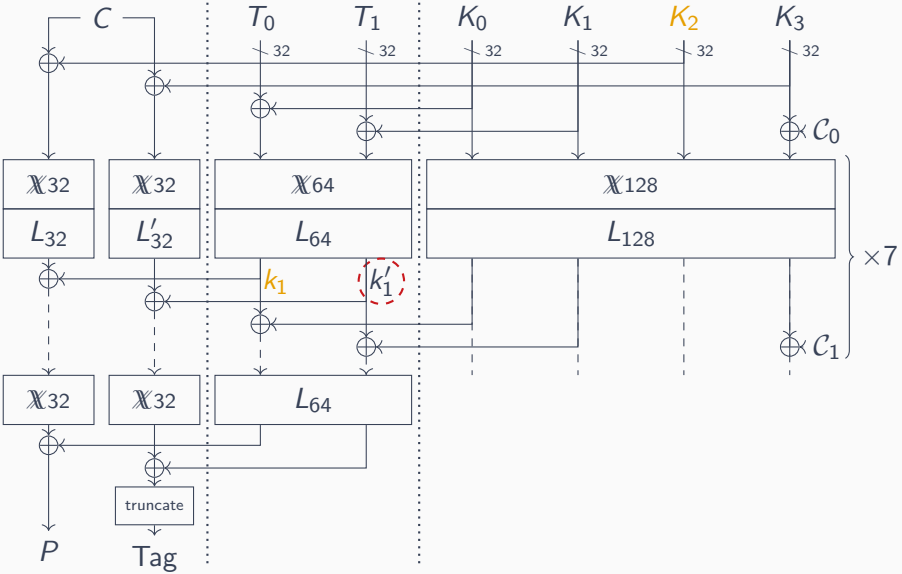
Master Key Recovery



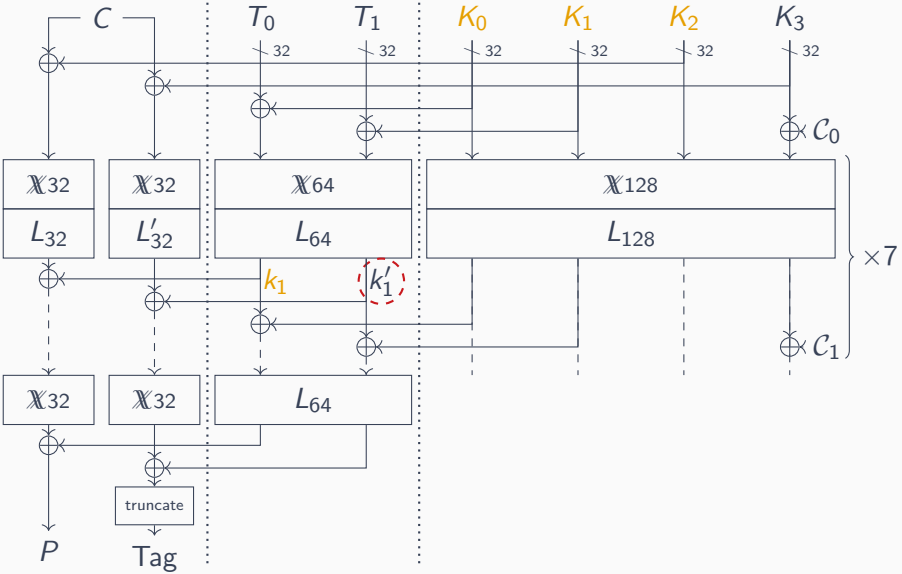
Master Key Recovery



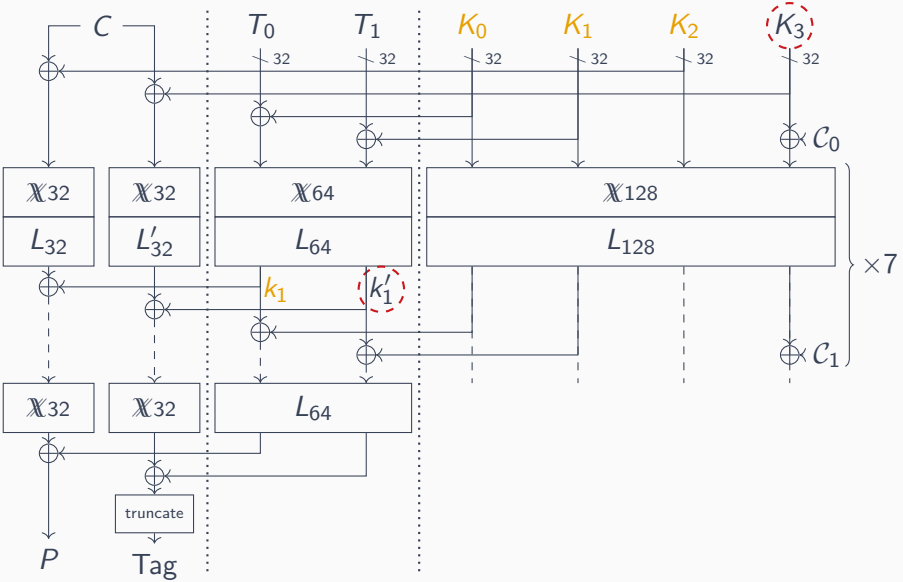
Master Key Recovery



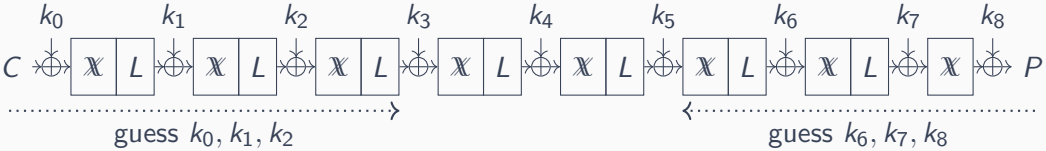
Master Key Recovery



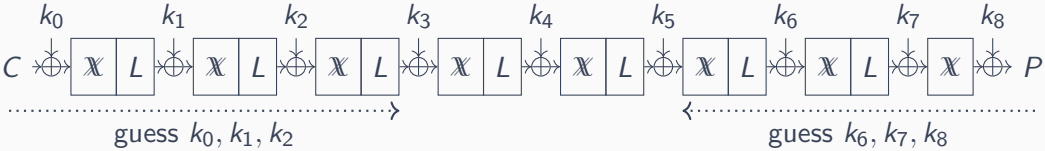
Master Key Recovery



Extendable to the Full 8 Rounds?

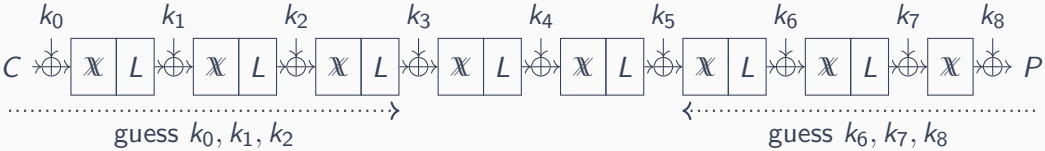


Extendable to the Full 8 Rounds?



- Enough to match on a single bit (per plaintext-ciphertext pair)

Extendable to the Full 8 Rounds?



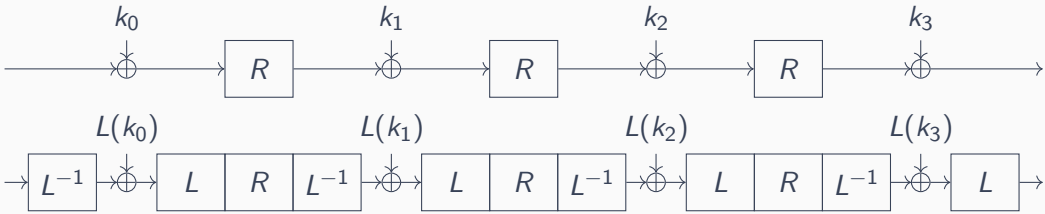
- Enough to match on a single bit (per plaintext-ciphertext pair)
- How much key material of k_3, k_4, k_5 do we need to know to match on one bit?

Meaning of Bits

Meaning of Bits

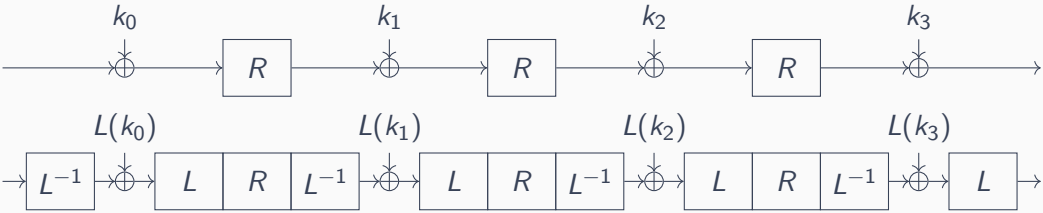
- Chose arbitrary linear permutation L

Meaning of Bits



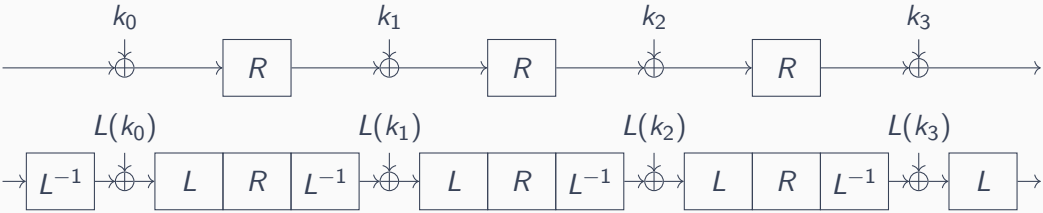
- Chose arbitrary linear permutation L

Meaning of Bits



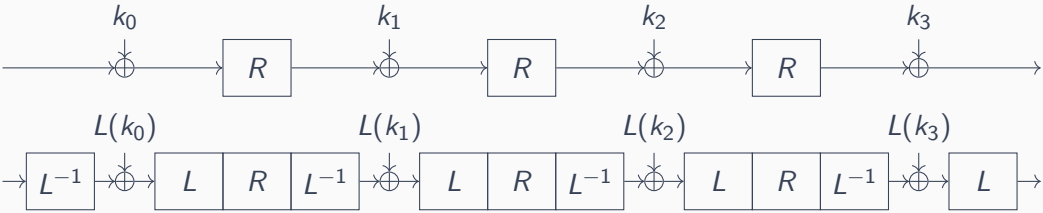
- Chose arbitrary linear permutation L
- Security independent of basis used to represent cipher!

Meaning of Bits



- Chose arbitrary linear permutation L
- Security independent of basis used to represent cipher!
- Hence, *bit* actually means *linear combination of bits*

Meaning of Bits



- Chose arbitrary linear permutation L
- Security independent of basis used to represent cipher!
- Hence, *bit* actually means *linear combination of bits*
- Represent bits of $x \in \mathbb{F}_2^n$ as $v^\mathsf{T} x = \sum_i v_i x_i$ for mask $v \in \mathbb{F}_2^n$

Dependencies of a Single Output Bit

- Want to compute $v^\mathsf{T} R(x)$ for some *mask* $v \in \mathbb{F}_2^n$

Dependencies of a Single Output Bit

- Want to compute $v^\intercal R(x)$ for some *mask* $v \in \mathbb{F}_2^n$
- Correlation: $C^R_{v,u} = 2^{-n} \sum_x (-1)^{u^\intercal x + v^\intercal R(x)}$ (Walsh-Hadamard transform)

Dependencies of a Single Output Bit

- Want to compute $v^\intercal R(x)$ for some *mask* $v \in \mathbb{F}_2^n$
- Correlation: $C^R_{v,u} = 2^{-n} \sum_x (-1)^{u^\intercal x + v^\intercal R(x)}$ (Walsh-Hadamard transform)
- Well known: $(-1)^{v^\intercal R(x)} = \sum_u C^R_{v,u} (-1)^{u^\intercal x}$ (inverse)

Dependencies of a Single Output Bit

- Want to compute $v^\intercal R(x)$ for some *mask* $v \in \mathbb{F}_2^n$
- Correlation: $C^R_{v,u} = 2^{-n} \sum_x (-1)^{u^\intercal x + v^\intercal R(x)}$ (Walsh-Hadamard transform)
- Well known: $(-1)^{v^\intercal R(x)} = \sum_u C^R_{v,u} (-1)^{u^\intercal x}$ (inverse)
- Enough to know $u^\intercal x$ for all $u \in U_v = \{u \mid C^R_{v,u} \neq 0\}$

Dependencies of a Single Output Bit

- Want to compute $v^\mathsf{T} R(x)$ for some *mask* $v \in \mathbb{F}_2^n$
- Correlation: $C^R_{v,u} = 2^{-n} \sum_x (-1)^{u^\mathsf{T} x + v^\mathsf{T} R(x)}$ (Walsh-Hadamard transform)
- Well known: $(-1)^{v^\mathsf{T} R(x)} = \sum_u C^R_{v,u} (-1)^{u^\mathsf{T} x}$ (inverse)
- Enough to know $u^\mathsf{T} x$ for all $u \in U_v = \{u \mid C^R_{v,u} \neq 0\}$

  $\iff$ knowing $u^\mathsf{T} x$ for all $u \in \mathrm{span}(U_v)$

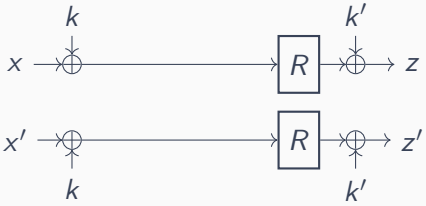
Dependencies of a Single Output Bit

- Want to compute $v^\mathsf{T} R(x)$ for some *mask* $v \in \mathbb{F}_2^n$
- Correlation: $C^R_{v,u} = 2^{-n} \sum_x (-1)^{u^\mathsf{T} x + v^\mathsf{T} R(x)}$ (Walsh-Hadamard transform)
- Well known: $(-1)^{v^\mathsf{T} R(x)} = \sum_u C^R_{v,u} (-1)^{u^\mathsf{T} x}$ (inverse)
- Enough to know $u^\mathsf{T} x$ for all $u \in U_v = \{u \mid C^R_{v,u} \neq 0\}$

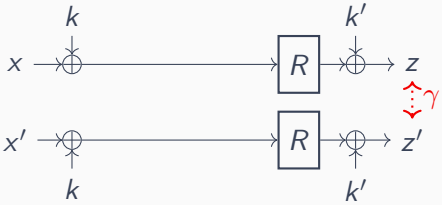
  $\iff$ knowing $u^\mathsf{T} x$ for all $u \in \operatorname{span}(U_v)$

  $\iff$ knowing $u_i^\mathsf{T} x$ for a basis $u_0, \ldots, u_{b-1}$ of $\operatorname{span}(U_v)$

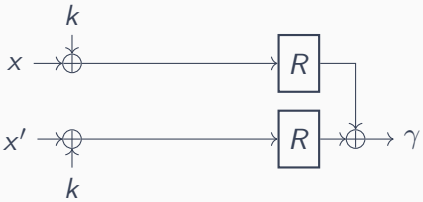
Dependencies of Differences



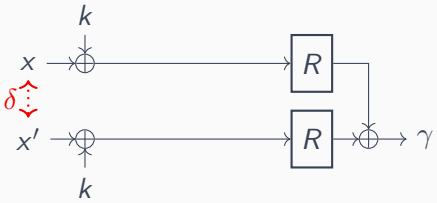
Dependencies of Differences



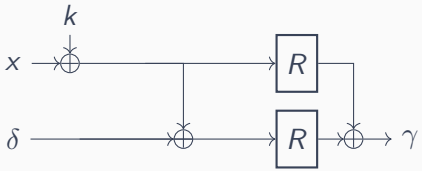
Dependencies of Differences



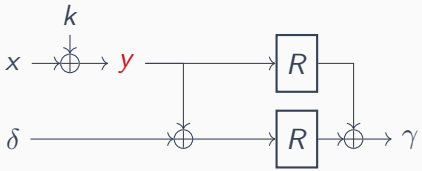
Dependencies of Differences



Dependencies of Differences

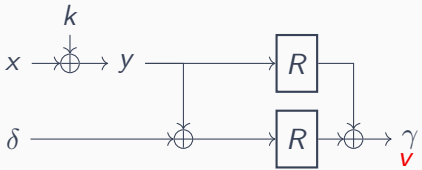


Dependencies of Differences



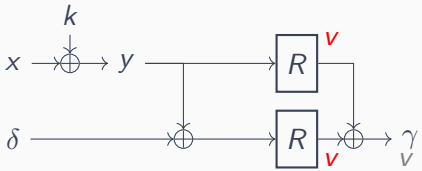
Dependencies of Differences

- Want to compute $v^{\mathsf{T}}\gamma$



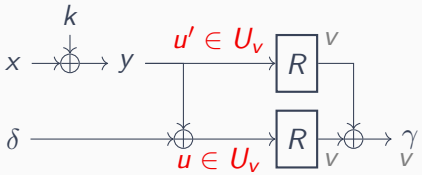
Dependencies of Differences

- Want to compute $v^{\mathsf{T}}\gamma$



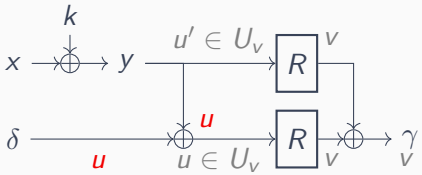
Dependencies of Differences

- Want to compute $v^{\mathsf{T}}\gamma$

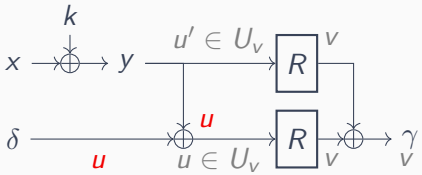


Dependencies of Differences

- Want to compute $v^{\mathsf{T}}\gamma$



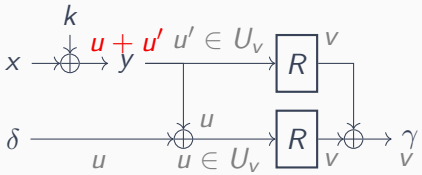
Dependencies of Differences



- Want to compute $v^{\mathsf{T}}\gamma$
- Dependencies on

$$\delta\text{: basis of span}\left(\quad U_v \right)$$

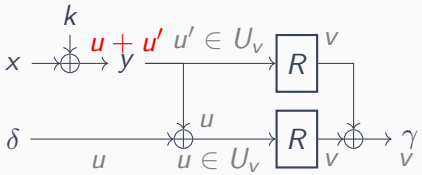
Dependencies of Differences



- Want to compute $v^{\mathsf{T}}\gamma$
- Dependencies on

  δ: basis of span (U_v)

Dependencies of Differences

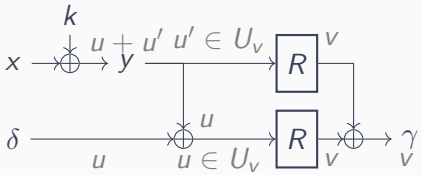


- Want to compute $v^{\mathsf{T}}\gamma$
- Dependencies on

  δ: basis of span (U_v)

  y: basis of span ($U_v + U_v$)

Dependencies of Differences

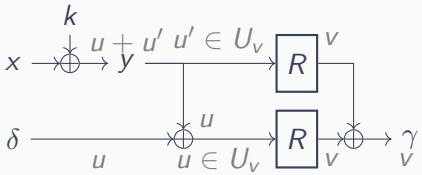


- Want to compute $v^{\mathsf{T}}\gamma$
- Dependencies on

  δ: basis of span $(\quad U_v)$

  y: basis of span $(\quad U_v + U_v \quad)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general

Dependencies of Differences

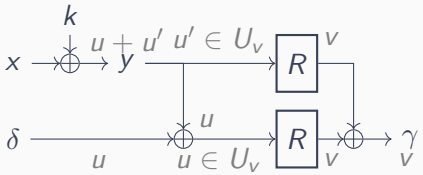


- Want to compute $v^{\mathsf{T}}\gamma$ for $v \in V$
- Dependencies on

  δ: basis of span(U_v)

  y: basis of span$(U_v + U_v)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general

Dependencies of Differences

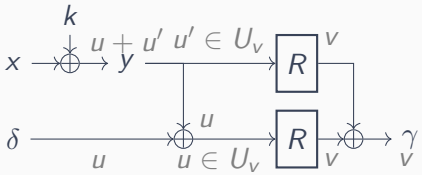


- Want to compute $v^{\mathsf{T}}\gamma$ for $v \in V$
- Dependencies on

  δ: basis of span(U_v)

  y: basis of span$(U_v + U_v)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$

Dependencies of Differences

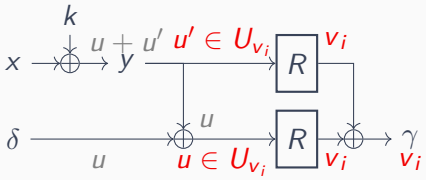


- Want to compute $v_i^{\mathsf{T}}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span (U_v)

  y: basis of span ($U_v + U_v$)
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$

Dependencies of Differences

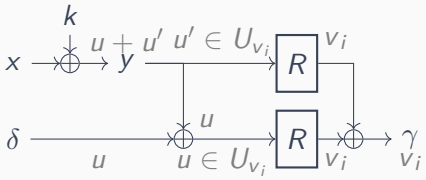


- Want to compute $v_i{}^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span $(\quad U_v)$

  y: basis of span $(\quad U_v + U_v \quad)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$

Dependencies of Differences

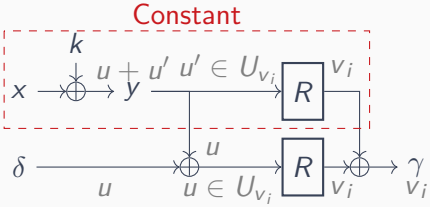


- Want to compute ${v_i}^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span $\left(\bigcup_i U_{v_i}\right)$

  y: basis of span $\left(\bigcup_i (U_{v_i} + U_{v_i})\right)$
- Note: $\operatorname{span}(U_v + U_v) \subseteq \operatorname{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\operatorname{span}(V)$

Dependencies of Differences

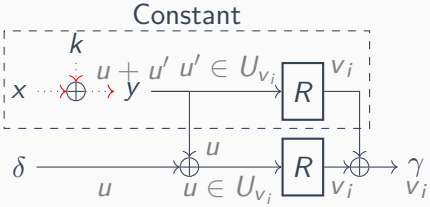


- Want to compute $v_i^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span $\left(\bigcup_i U_{v_i}\right)$

  y: basis of span $\left(\bigcup_i \left(U_{v_i} + U_{v_i}\right)\right)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$

Dependencies of Differences

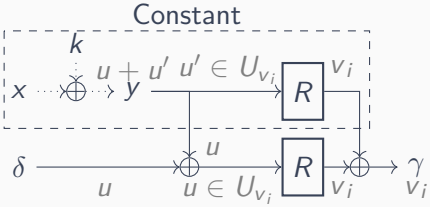


- Want to compute ${v_i}^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span $\left(\bigcup_i U_{v_i}\right)$

  y: basis of span $\left(\bigcup_i \left(U_{v_i} + U_{v_i}\right)\right)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$
- Guess state y instead of key k

Dependencies of Differences

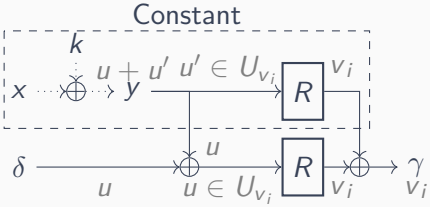


- Want to compute ${v_i}^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

  δ: basis of span $\left(\bigcup_i U_{v_i}\right)$

  y: basis of span $\left(\bigcup_i \left(U_{v_i} + U_{v_i}\right)\right)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$
- Guess state y instead of key k
- Can be iterated over multiple rounds

Dependencies of Differences



- Want to compute ${v_i}^\mathsf{T}\gamma$ for $v_i \in V$
- Dependencies on

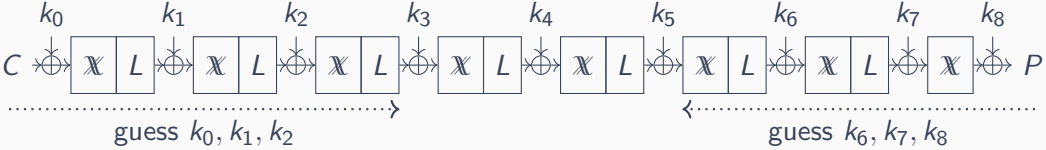
  δ: basis of span $\left(\bigcup_i U_{v_i}\right)$

  y: basis of span $\left(\bigcup_i \left(U_{v_i} + U_{v_i}\right)\right)$
- Note: $\mathrm{span}(U_v + U_v) \subseteq \mathrm{span}(U_v)$, but equality does *not* hold in general
- $v_0, \ldots, v_{b-1}$ basis of $\mathrm{span}(V)$
- Guess state y instead of key k
- Can be iterated over multiple rounds
- Guessing y reduces dependencies when restricting δ

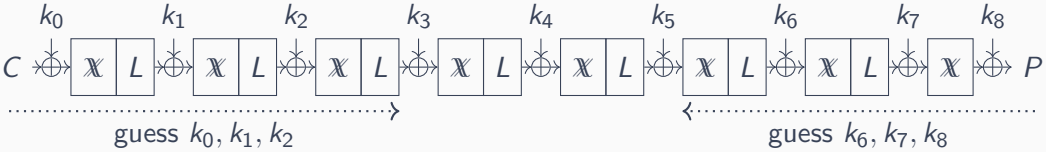
State Dependencies for 2-Round ChiLow

Block Size	Number of Matching Bits	State Dependencies (/Bits)
32	1	17
	2	23
	3	27
40	1	17
	2	24
	3	28

A Known Plaintext Attack on Full ChiLow-32

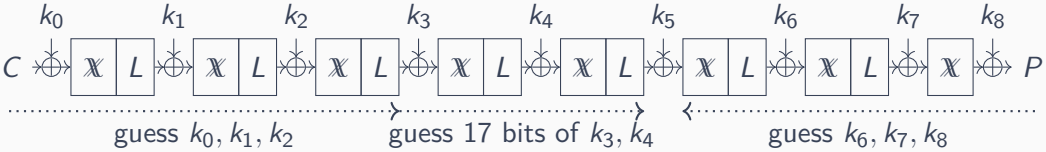


A Known Plaintext Attack on Full ChiLow-32



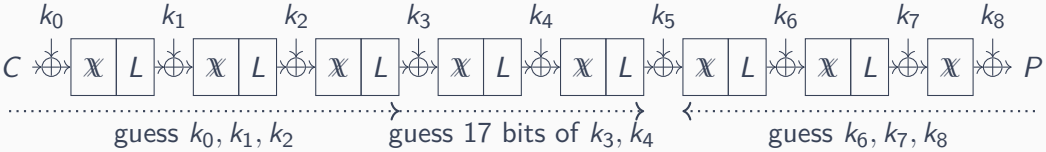
- Optimal time complexity by matching on one bit and using $160 \approx 2^{7.34}$ known plaintext-ciphertext pairs

A Known Plaintext Attack on Full ChiLow-32



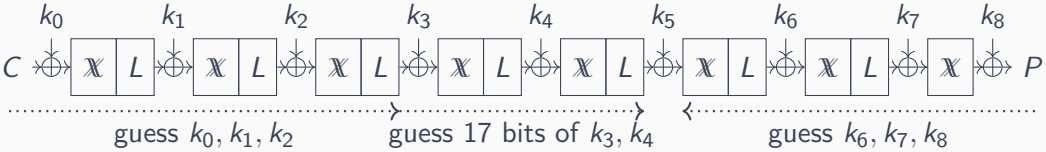
- Optimal time complexity by matching on one bit and using $160 \approx 2^{7.34}$ known plaintext-ciphertext pairs

A Known Plaintext Attack on Full ChiLow-32



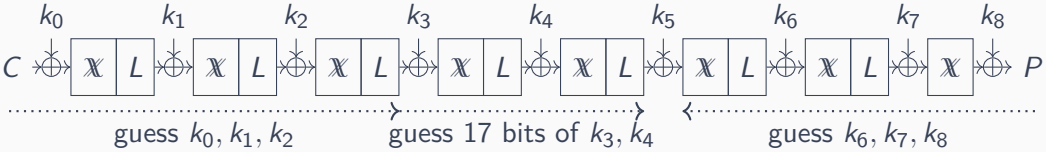
- Optimal time complexity by matching on one bit and using $160 \approx 2^{7.34}$ known plaintext-ciphertext pairs
- Guess $3 \cdot 32 + 17 = 113$ bits from the ciphertext side

A Known Plaintext Attack on Full ChiLow-32



- Optimal time complexity by matching on one bit and using $160 \approx 2^{7.34}$ known plaintext-ciphertext pairs
- Guess $3 \cdot 32 + 17 = 113$ bits from the ciphertext side
- Guess $3 \cdot 32 = 96$ bits from the plaintext side

A Known Plaintext Attack on Full ChiLow-32



- Optimal time complexity by matching on one bit and using $160 \approx 2^{7.34}$ known plaintext-ciphertext pairs
- Guess $3 \cdot 32 + 17 = 113$ bits from the ciphertext side
- Guess $3 \cdot 32 = 96$ bits from the plaintext side
- Complexities: $2^{120.34}$ (time), $2^{98.32}$ (memory) and 160 (data)

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$
- $v^\mathsf{T}(R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$
- $v^\mathsf{T}(R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$
- $v^\mathsf{T}(R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$
- $v^\mathsf{T}(R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$
- Restricting input difference can reduce state dependencies

Reducing State Dependencies

- Example: $v^\mathsf{T} R(x) = x_0 + x_1 x_2 + x_2$
- $v^\mathsf{T}(R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$
- Restricting input difference can reduce state dependencies
- Idea: compute state dependencies for derivatives $x \mapsto R(x) + R(x+\delta)$ instead of R

Reducing State Dependencies

- Example: $v^\intercal R(x) = x_0 + x_1 x_2 + x_2$
- $v^\intercal (R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$
- Restricting input difference can reduce state dependencies
- Idea: compute state dependencies for derivatives $x \mapsto R(x) + R(x+\delta)$ instead of R
- Propagate truncated differences Δ, modeled as affine spaces

Reducing State Dependencies

- Example: $v^\intercal R(x) = x_0 + x_1 x_2 + x_2$
- $v^\intercal (R(x) + R(x+\delta)) = \delta_0 + x_1\delta_2 + x_2\delta_1 + \delta_1\delta_2 + \delta_2$
- Restricting input difference can reduce state dependencies
- Idea: compute state dependencies for derivatives $x \mapsto R(x) + R(x+\delta)$ instead of R
- Propagate truncated differences Δ, modeled as affine spaces
- Enough to consider basis $\delta_0, \ldots, \delta_{b-1}$ and offset α of Δ

State Dependencies for Computing the Output Difference of 2-Round ChiLow

Block Size	Dimension	Non-constant Bits	State Dependency (/Bits)
32	6	2 – 7	33
	7	2 – 8	35
	8	2 – 9	37
40	6	10 – 15	40
	7	10 – 16	43
	8	10 – 17	45

An Optimized Chosen Ciphertext Attack on Full ChiLow-32

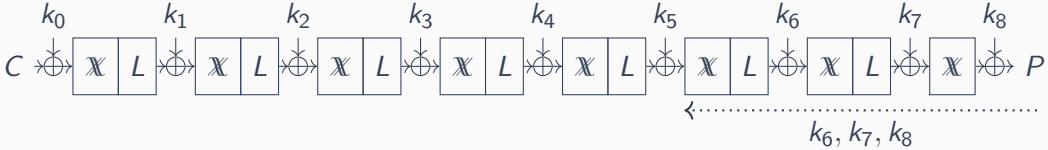
- Use 6-dimensional space (requiring 33 state bits over 2 rounds)

An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)

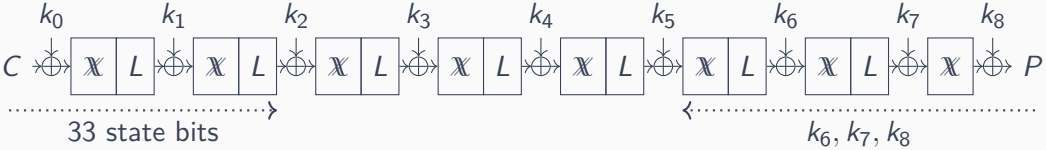
An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



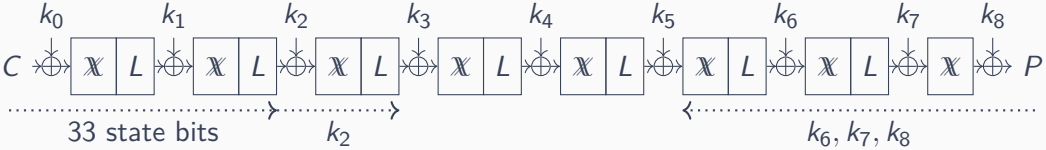
An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



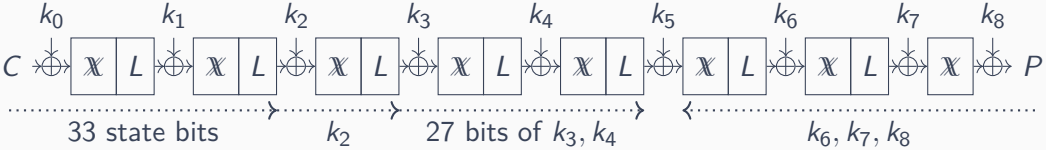
An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



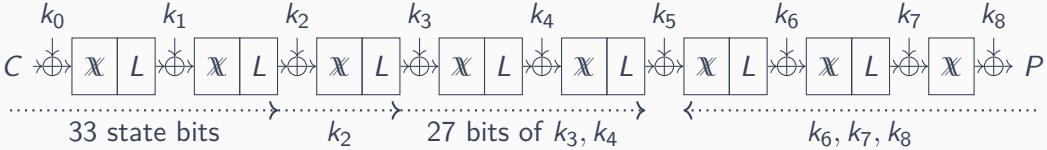
An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



An Optimized Chosen Ciphertext Attack on Full ChiLow-32

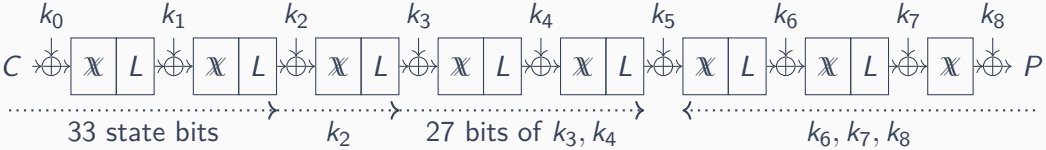
- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



- Guess $33 + 32 + 27 = 92$ bits from the ciphertext side

An Optimized Chosen Ciphertext Attack on Full ChiLow-32

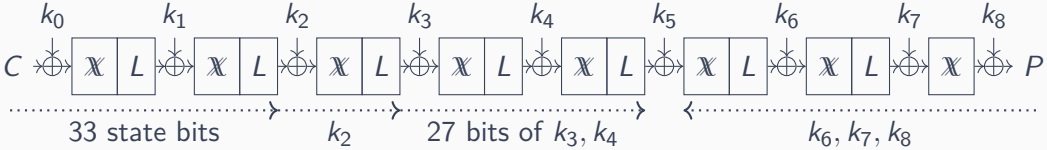
- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



- Guess $33 + 32 + 27 = 92$ bits from the ciphertext side
- Guess $3 \cdot 32 = 96$ bits from the plaintext side

An Optimized Chosen Ciphertext Attack on Full ChiLow-32

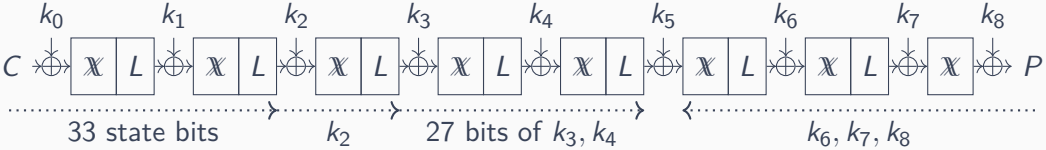
- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



- Guess $33 + 32 + 27 = 92$ bits from the ciphertext side
- Guess $3 \cdot 32 = 96$ bits from the plaintext side
- Master key recovery more complicated

An Optimized Chosen Ciphertext Attack on Full ChiLow-32

- Use 6-dimensional space (requiring 33 state bits over 2 rounds)
- Matching on 3 bits (requiring 27 state bits over 2 rounds)



- Guess $33 + 32 + 27 = 92$ bits from the ciphertext side
- Guess $3 \cdot 32 = 96$ bits from the plaintext side
- Master key recovery more complicated
- Complexities: $2^{102.09}$ (time), $2^{94.56}$ (memory) and 2^6 (data)

Overview of Results

Variant	Rounds	Time	Memory	Data	Type	Ref.
ChiLow-32	4	2^{126}	–	–	MitM	[BDG+25]
	5	2^{32}	$2^{28.65}$	$2^{18.58}$ CC	Cube	[AHH+26]
	6	2^{34}	$2^{49.28}$	$2^{33.58}$ CC	Cube	[AHH+16]
	7	$2^{121.75}$	–	80 CC	Integral	[AHH+16]
	8	$2^{120.34}$	$2^{98.32}$	160 KP	MitM	This Work
	8	$2^{102.09}$	$2^{94.56}$	64 CC	MitM	This Work
	9	2^{118}	$2^{106.68}$	256 CC	MitM	This Work
ChiLow-40	4	2^{126}	–	–	MitM	[BDG+25]
	8	$2^{122.32}$	$2^{114.68}$	256 CP	MitM	This Work

Overview of Results

Variant	Rounds	Time	Memory	Data	Type	Ref.
ChiLow-32	4	2^{126}	–	–	MitM	[BDG+25]
	5	2^{32}	$2^{28.65}$	$2^{18.58}$ CC	Cube	[AHH+26]
	6	2^{34}	$2^{49.28}$	$2^{33.58}$ CC	Cube	[AHH+16]
	7	$2^{121.75}$	–	80 CC	Integral	[AHH+16]
	8	$2^{120.34}$	$2^{98.32}$	160 KP	MitM	This Work
	8	$2^{102.09}$	$2^{94.56}$	64 CC	MitM	This Work
	9	2^{118}	$2^{106.68}$	256 CC	MitM	This Work
ChiLow-40	4	2^{126}	–	–	MitM	[BDG+25]
	8	$2^{122.32}$	$2^{114.68}$	256 CP	MitM	This Work

Overview of Results

Variant	Rounds	Time	Memory	Data	Type	Ref.
ChiLow-32	4	2^{126}	–	–	MitM	[BDG+25]
	5	2^{32}	$2^{28.65}$	$2^{18.58}$ CC	Cube	[AHH+26]
	6	2^{34}	$2^{49.28}$	$2^{33.58}$ CC	Cube	[AHH+16]
	7	$2^{121.75}$	–	80 CC	Integral	[AHH+16]
	8	$2^{120.34}$	$2^{98.32}$	160 KP	MitM	This Work
	8	$2^{102.09}$	$2^{94.56}$	64 CC	MitM	This Work
	9	2^{118}	$2^{106.68}$	256 CC	MitM	This Work
ChiLow-40	4	2^{126}	–	–	MitM	[BDG+25]
	8	$2^{122.32}$	$2^{114.68}$	256 CP	MitM	This Work

Overview of Results

Variant	Rounds	Time	Memory	Data	Type	Ref.
ChiLow-32	4	2^{126}	–	–	MitM	[BDG+25]
	5	2^{32}	$2^{28.65}$	$2^{18.58}$ CC	Cube	[AHH+26]
	6	2^{34}	$2^{49.28}$	$2^{33.58}$ CC	Cube	[AHH+16]
	7	$2^{121.75}$	–	80 CC	Integral	[AHH+16]
	8	$2^{120.34}$	$2^{98.32}$	160 KP	MitM	This Work
	8	$2^{102.09}$	$2^{94.56}$	64 CC	MitM	This Work
	9	2^{118}	$2^{106.68}$	256 CC	MitM	This Work
ChiLow-40	4	2^{126}	–	–	MitM	[BDG+25]
	8	$2^{122.32}$	$2^{114.68}$	256 CP	MitM	This Work

Possible Countermeasures

- Increase the number of rounds

Possible Countermeasures

- Increase the number of rounds
  - Increases latency

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)
  - Using more rounds might be cheaper

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)
  - Using more rounds might be cheaper
- Increase the state size

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)
  - Using more rounds might be cheaper
- Increase the state size
  - Possible for ChiLow-40 only

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)
  - Using more rounds might be cheaper
- Increase the state size
  - Possible for ChiLow-40 only
  - Increases ciphertext size

Possible Countermeasures

- Increase the number of rounds
  - Increases latency
- Increase the diffusion per round
  - Likely to increase latency (per round)
  - Using more rounds might be cheaper
- Increase the state size
  - Possible for ChiLow-40 only
  - Increases ciphertext size

Thank you for your attention!